



Retro Snake Firmware for Arduino

A robust, state-machine-driven Snake game implementation for the ATmega328P (Arduino Nano/Uno) using the Nokia 5110 LCD.



Firmware Architecture

State Machine Design

The firmware uses an enum state machine to cleanly separate logic. Only one state runs per loop() cycle.

```
enum State {MENU, GAME, HELP, DIFFICULTY, GAME_OVER, SCORES, ABOUT};  
State state = MENU;
```

```
void loop() {  
  switch(state){  
    case MENU:    drawMenu();    handleMenuInput(); break;  
    case GAME:    gameLoop();    break;  
    case HELP:    drawHelp();    break;  
    case DIFFICULTY: drawDifficulty(); break;  
    case GAME_OVER: drawGameOver(); break;  
    case SCORES:  drawHighScores(); break;  
    case ABOUT:  drawAbout();    break;  
  }  
}
```



Input Handling & Debounce

Buttons are read using a **non-blocking time delta** to prevent bounce without delay().

```
bool isButtonPressed(int pin) {  
  if (digitalRead(pin) == LOW) {  
    unsigned long now = millis();  
    if (now - lastButtonPress >= DEBOUNCE_MS) {  
      lastButtonPress = now;  
      return true;  
    }  
  }  
  return false;  
}
```

- **Logic:** INPUT_PULLUP + GND press = LOW signal.
- **Debounce:** DEBOUNCE_MS = 180 ensures clean transitions.
- **Directional Lock:** Inputs in-game ignore reverse commands (if(dy == 0) prevents 180° U-turn crashes).



Menu System

- **Viewport Scrolling:** Displays 4 of 6 menu items. Automatically scrolls to keep the cursor centered.
- **Memory Optimization:** Uses F() macro for all string literals to save ≈ 150 bytes SRAM.
- **Dynamic Toggles:** SOUND menu item instantly reflects soundOn state on screen.



Game Loop Mechanics

Runs strictly on a timer-based speedDelay to control snake velocity.

1. handleGameInput() → Debounced directional changes
2. millis() - lastMove > speedDelay → Advances snake
3. updateSnake() → Shifts body segments, updates head
4. checkCollision() → Wall bounds + self-intersection
5. checkFood() → Score++, growth, flash effect, spawn new food, play sfxEat()



EEPROM High Score System

- **Storage:** Top 4 scores stored in 8 bytes (EEPROM_ADDR 0-7).
- **Loading:** EEPROM.get() loads on boot; validates $0 \leq \text{score} \leq 9999$.
- **Updating:** On GAME_OVER, updateHighScores() inserts new scores in descending order, shifts older scores down, and saves back to EEPROM.
- **Feedback:** Plays sfxHighScore() only when hitting #1 position.



8-Bit Sound Design

All SFX functions check if(soundOn) before playing. Synthesized via tone() on D6.

Function	Trigger	Pattern
sfxBootStart()	Boot	440Hz →
sfxLoadTick()	Loading Bar	Random 600-900H
sfxReady()	Post-Loading	1200Hz →
sfxTitle()	Main Menu Intro	4-note arpeggio
sfxMenuMove()	Menu Navigation	Short 800Hz blip
sfxEat()	Food Collision	1000Hz →
sfxWall()	Crash	Low 150Hz buzz
sfxGameOver()	Death Screen	Descending scale (600 →)
sfxHighScore()	New #1 Score	1500Hz → fanfare
sfxBack()	Return to Menu	Mid → Low drop

📦 Setup & Usage

- 1. **Install Libraries** (Arduino IDE → Library Manager):
 - Adafruit GFX Library
 - Adafruit PCD8544 Nokia 5110 LCD Library
- 2. **Open** v3_full.ino in Arduino IDE.
- 3. **Configure Board:** Tools → Board: Arduino Nano, Processor: ATmega328P.
- 4. **Upload:** Game auto-runs startup animation on first power.

🔧 Customization Guide

Parameter	File Location	Effect
CELL size	Line 9	Snake/food size (default: 4px)
DEBOUNCE_MS	Line 38	Button press filtering time
speedDelay values	updateDifficulty()	Easy(180), Medium(120), Hard(70), Extreme(40) ms
MAX_SNAKE	Line 43	Max body segments (default: 30)
display.setContrast()	setup()	LCD contrast (58 recommended)
EEPROM_ADDR	Line 40	Base address for scores (change if storing other data)

📄 Firmware Versions

Folder	Size (Flash/RAM)	Description
v1_basic	≈	Core movement, no SFX, no menu. Best for debugging hardware.
v2_lighter	≈	Stripped UI, optimized loops, minimal RAM footprint.
v3_full	≈	Recommended. Full features: SFX, scrolling menu, EEPROM, 3-page About.
startup_animation	≈	Standalone boot sequence

		for hardware verification.
--	--	----------------------------



Contributing

1. Fork & create a feature branch.
2. Keep .ino files self-contained for Arduino IDE compatibility.
3. Update CHANGELOG.md with hardware/firmware changes.
4. Submit PR with clear commit messages (feat:, fix:, opt:).